

Appendix A. Answers to Review Questions

Chapter 4, Cluster Installation and Upgrade

1. List the cluster nodes using the following command. You should see:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	2m20s	v1.32.2
minikube-m02	Ready	<none>	2m10s	v1.32.2
minikube-m03	Ready	<none>	2m3s	v1.32.2
minikube-m04	Ready	<none>	116s	v1.32.2

Schedule the Pod using the following imperative command:

```
$ kubectl run nginx --image=nginx:1.27.4-alpine
```

You can identify the node the Pod is running, as shown. The Pod is running on the node named `minikube-m02`:

```
$ kubectl get pod nginx -o jsonpath='{.spec.nodeName}'
minikube-m02
```

Use the `kubectl drain` command to evict all Pods on a node. Here, the node name is `minikube-m02`. You'll need to use the `--ignore-daemonsets` flag to only delete Pods that are not managed by a `DaemonSet`, and the `--force` flag to delete Pods without a controller:

```
$ kubectl drain minikube-m02 --ignore-daemonsets --force
evicting pod default/nginx
pod/nginx evicted
node/minikube-m02 drained
```

2. The solution to this sample exercise requires a lot of manual steps. The following commands do not render their output.

Open an interactive shell to the control plane node using Vagrant:

```
$ vagrant ssh kube-control-plane
```

Upgrade `kubeadm` to version 1.32.2 and apply it:

```
$ sudo apt-mark unhold kubeadm && sudo apt-get update && sudo apt-get \  
install -y kubeadm=1.32.2-1.1 && sudo apt-mark hold kubeadm  
$ sudo kubeadm upgrade apply v1.32.2
```

Drain the node, upgrade the kubelet and `kubect1` , restart the kubelet, and uncordon the node:

```
$ kubectl drain kube-control-plane --ignore-daemonsets  
$ sudo apt-get update && sudo apt-get install -y \  
--allow-change-held-packages kubelet=1.32.2-1.1 kubect1=1.32.2-1.1  
$ sudo systemctl daemon-reload  
$ sudo systemctl restart kubelet  
$ kubectl uncordon kube-control-plane
```

The version of the node should now say v1.32.2. Exit the node:

```
$ kubectl get nodes  
$ exit
```

Open an interactive shell to the worker node using Vagrant:

```
$ vagrant ssh kube-worker-1
```

Upgrade `kubeadm` to version 1.32.2 and apply it to the node:

```
$ sudo apt-get update && sudo apt-get install -y \  
--allow-change-held-packages kubeadm=1.32.2-1.1  
$ sudo kubeadm upgrade node
```

Drain the node, upgrade the kubelet and `kubect1` , restart the kubelet, and uncordon the node:

```
$ kubectl drain kube-worker-1 --ignore-daemonsets  
$ sudo apt-get update && sudo apt-get install -y \
```

```
--allow-change-held-packages kubelet=1.32.2-1.1 kubect1=1.32.2-1.1
$ sudo systemctl daemon-reload
$ sudo systemctl restart kubelet
$ kubect1 uncordon kube-worker-1
```

The version of the node should now say v1.32.2. Exit the node:

```
$ kubect1 get nodes
$ exit
```

Chapter 5, Backing Up and Restoring etcd

1. Open an interactive shell to the control plane node using Vagrant:

```
$ vagrant ssh kube-control-plane
```

The etcd Pod runs in the `kube-system` namespace. Identify the Pod by listing all Pods in the namespace. You should find the Pod named `etcd-kube-control-plane`:

```
$ kubect1 get pods -n kube-system
```

The etcd Pod only runs a single container. You can render the details of the Pod using the `kubect1 get` or `kubect1 describe` command. To select only the container image, use the following JSONPath:

```
$ kubect1 get pod etcd-kube-control-plane -n kube-system \
-o jsonpath="{.spec.containers[0].image}"
```

The container uses the image `registry.k8s.io/etcd:3.5.16-0`. The etcd version used is 3.5.16. Write the version to the file named `etcd-version.txt`. You can view all released etcd versions in the corresponding [GitHub repository](#):

```
$ echo "3.5.16" > etcd-version.txt
```

Exit the node:

```
$ exit
```

2. The solution to this sample exercise requires a lot of manual steps. The following commands do not render their output.

Open an interactive shell to the control plane node using Vagrant:

```
$ vagrant ssh kube-control-plane
```

Determine the parameters of the Pod `etcd-kube-control-plane` by describing it. Use the correct parameter values to create a snapshot file:

```
$ kubectl describe pod etcd-kube-control-plane -n kube-system  
$ sudo ETCDCTL_API=3 etcdctl --cacert=/etc/kubernetes/pki/etcd/ca.crt \  
  --cert=/etc/kubernetes/pki/etcd/server.crt \  
  --key=/etc/kubernetes/pki/etcd/server.key snapshot save /opt/etcd.bak
```

Restore the backup from the snapshot file. Edit the etcd YAML manifest and change the value of `spec.volumes.hostPath.path` for the volume named `etcd-data` to `/var/bak`:

```
$ sudo ETCDCTL_API=3 etcdctl --data-dir=/var/bak snapshot restore \  
  /opt/etcd.bak  
$ sudo vim /etc/kubernetes/manifests/etcd.yaml
```

After a short while, the Pod `etcd-kube-control-plane` should transition back into the `Running` status. Exit the node:

```
$ kubectl get pod etcd-kube-control-plane -n kube-system  
$ exit
```

Chapter 6, Authentication, Authorization, and Admission Control

1. Use the `kubectl create clusterrole` command to create the Role imperatively:

```
$ kubectl create clusterrole service-view --verb=get,list \  
  --resource=services
```

If you'd rather start with the YAML file, use content shown in the file *service-view-clusterrole.yaml*:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: service-view
rules:
- apiGroups: ["" ]
  resources: ["services"]
  verbs: ["get", "list"]
```

Create the ClusterRole from the YAML file:

```
$ kubectl apply -f service-view-clusterrole.yaml
```

Create the development namespace:

```
$ kubectl create namespace development
```

Use the `kubectl create rolebinding` command to create the RoleBinding imperatively:

```
$ kubectl create rolebinding ellasmith-service-view --user=ellasmith \
--clusterrole=service-view -n development
```

The declarative approach of the RoleBinding could look like the one in the file *ellasmith-service-view-rolebinding.yaml*:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ellasmith-service-view
  namespace: development
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: service-view
subjects:
- kind: User
  name: ellasmith
  apiGroup: rbac.authorization.k8s.io
```

Create the RoleBinding from the YAML file:

```
$ kubectl apply -f ellasmith-service-view-rolebinding.yaml
```

Use the `kubectl create clusterrole` command to create the ClusterRole imperatively:

```
$ kubectl create clusterrole combined \
  --aggregation-rule="rbac.cka.cncf.com/aggregate=true"
```

If you'd rather start with the YAML file, use content shown in the file *combined-clusterrole.yaml*:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: combined
aggregationRule:
  clusterRoleSelectors:
  - matchLabels:
      rbac.cka.cncf.com/aggregate: "true"
rules: [ ]
```

Create the ClusterRole from the YAML file:

```
$ kubectl apply -f combined-clusterrole.yaml
```

Rendering the rules shows an empty list, as no ClusterRoles were selected by label selection:

```
$ kubectl describe clusterrole combined
Name:          combined
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  -----  -

```

Use the `kubectl create clusterrole` command to create the ClusterRole imperatively. The command does not provide an option for adding a label. Therefore, generating the YAML manifest using the dry-run option is a good start:

```
$ kubectl create clusterrole deployment-modify \
  --verb=create,delete,patch,update --resource=deployments \
  --dry-run=client -o yaml > deployment-modify-clusterrole.yaml
```

Modify the content shown in the file *deployment-modify-clusterrole.yaml*:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: deployment-modify
  labels:
    rbac.cka.cncf.com/aggregate: "true"
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["create", "delete", "patch", "update"]
```

Create the ClusterRole from the YAML file:

```
$ kubectl apply -f deployment-modify-clusterrole.yaml
```

Rendering the rules for the ClusterRole `combined` lists the rules defined by the ClusterRole `deployment-modify`:

```
$ kubectl describe clusterrole combined
```

```
Name:          combined
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources            Non-Resource URLs   Resource Names   Verbs
  -----
  deployments.apps     []                  []               [create \
  delete patch update]
```

Listing Services is allowed for the user `ellasmith` in the namespace `development`. This is taken care of by RoleBinding `ellasmith-service-view`:

```
$ kubectl auth can-i list services --as=ellasmith --namespace=development
yes
```

Write the output to the file *list-services-ellasmith.txt*:

```
$ echo 'yes' > list-services-ellasmith.txt
```

Watching Deployments is not allowed for the user `ellasmith` in the namespace `production`. The ClusterRole named `deployment-`

modify only allows the verbs `create`, `delete`, `patch`, and `update`:

```
$ kubectl auth can-i watch deployments --as=ellasmith --namespace=production
```

Write the output to the file *watch-deployments-ellasmith.txt*:

```
$ echo 'no' > watch-deployments-ellasmith.txt
```

Deployments are allowed to be created, deleted, patched, and updated.

2. First, create the namespace named `apps`. Then, we'll create the `ServiceAccount`:

```
$ kubectl create namespace apps
$ kubectl create serviceaccount api-access -n apps
```

Alternatively, you can use the declarative approach. Create the namespace from the definition in the file *apps-namespace.yaml*:

```
apiVersion: v1
kind: Namespace
metadata:
  name: apps
```

Create the namespace from the YAML file:

```
$ kubectl apply -f apps-namespace.yaml
```

Create a new YAML file called *api-serviceaccount.yaml* with the following contents:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api-access
  namespace: apps
```

Run the `apply` command to instantiate the `ServiceAccount` from the YAML file:


```
$ kubectl create -f api-serviceaccount.yaml
```

Use the `create clusterrole` command to create the ClusterRole imperatively:

```
$ kubectl create clusterrole api-clusterrole --verb=watch,list,get \
  --resource=pods
```

If you'd rather start with the YAML file, use content shown in the file *api-clusterrole.yaml*:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: api-clusterrole
rules:
- apiGroups: [ "" ]
  resources: [ "pods" ]
  verbs: [ "watch", "list", "get" ]
```

Create the ClusterRole from the YAML file:

```
$ kubectl apply -f api-clusterrole.yaml
```

Use the `create clusterrolebinding` command to create the ClusterRoleBinding imperatively:

```
$ kubectl create clusterrolebinding api-clusterrolebinding \
  --serviceaccount=apps:api-access --clusterrole=api-clusterrole
```

The declarative approach of the ClusterRoleBinding could look like the one in the file *api-clusterrolebinding.yaml*:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: api-clusterrolebinding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: api-clusterrole
subjects:
- apiGroup: ""
  kind: ServiceAccount
```

```
name: api-access
namespace: apps
```

Create the ClusterRoleBinding from the YAML file:

```
$ kubectl apply -f api-clusterrolebinding.yaml
```

Execute the `run` command to create the Pods in the different namespaces. You will need to create the namespace `rm` before you can instantiate the Pod `disposable`:

```
$ kubectl run operator --image=nginx:1.21.1 --restart=Never \
  --port=80 --serviceaccount=api-access -n apps
$ kubectl create namespace rm
$ kubectl run disposable --image=nginx:1.21.1 --restart=Never \
  -n rm
```

The following YAML manifest shows the `rm` namespace definition stored in the file *rm-namespace.yaml*:

```
apiVersion: v1
kind: Namespace
metadata:
  name: rm
```

The YAML representation of those Pods stored in the file *api-pods.yaml* could look as follows:

```
apiVersion: v1
kind: Pod
metadata:
  name: operator
  namespace: apps
spec:
  serviceAccountName: api-access
  containers:
  - name: operator
    image: nginx:1.21.1
    ports:
    - containerPort: 80
---
```

```
apiVersion: v1
kind: Pod
metadata:
  name: disposable
  namespace: rm
```

```
spec:
  containers:
  - name: disposable
    image: nginx:1.21.1
```

Create the namespace and Pods from the YAML files:

```
$ kubectl create -f rm-namespace.yaml
$ kubectl create -f api-pods.yaml
```

Determine the API server endpoint and the Secret access token of the ServiceAccount. You will need this information for making the API calls:

```
$ kubectl config view --minify -o \
  jsonpath='{.clusters[0].cluster.server}'
https://192.168.64.4:8443
$ kubectl get secret $(kubectl get serviceaccount api-access -n apps \
  -o jsonpath='{.secrets[0].name}') -o jsonpath='{.data.token}' -n apps
| base64 --decode
eyJhbGciOiJSUzI1NiIsImtpZCI6Ii1hOUhI...
```

Open an interactive shell to the Pod named `operator`:

```
$ kubectl exec operator -it -n apps -- /bin/sh
```

Emit API calls for listing all Pods and deleting the Pod `disposable` living in the `rm` namespace. You will find that while the `list` operation is permitted, the `delete` operation isn't:

```
# curl https://192.168.64.4:8443/api/v1/namespaces/rm/pods --header \
  "Authorization: Bearer eyJhbGciOiJSUzI1NiIsImtpZCI6Ii1hOUhI..." \
  --insecure
{
  "kind": "PodList",
  "apiVersion": "v1",
  ...
}
# curl -X DELETE https://192.168.64.4:8443/api/v1/namespaces \
  /rm/pods/disposable --header \
  "Authorization: Bearer eyJhbGciOiJSUzI1NiIsImtpZCI6Ii1hOUhI..." \
  --insecure
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {
```

```

    },
    "status": "Failure",
    "message": "pods \"disposable\" is forbidden: User \
    \"system:serviceaccount:apps:api-access\" cannot delete \
    resource \"pods\" in
    API group \"\" in the namespace \"rm\"",
    "reason": "Forbidden",
    "details": {
      "name": "disposable",
      "kind": "pods"
    },
    "code": 403
  }
}

```

Chapter 7, Operators and Custom Resource Definitions (CRDs)

1. Create the CRD from the provided URL:

```

$ kubectl apply -f https://raw.githubusercontent.com/mongodb/\
mongodb-kubernetes-operator/master/config/crd/bases/mongodbcommunity.\
mongodb.com_mongodbcommunity.yaml
customresourcedefinition.apiextensions.k8s.io/mongodbcommunity.\
mongodbcommunity.mongodb.com created

```

You can find the installed CRD named `mongodbcommunity.mongodbcommunity.mongodb.com` with the following command:

```

$ kubectl get crds
NAME                                                    CREATED AT
mongodbcommunity.mongodbcommunity.mongodb.com        2023-12-18T23:44:04Z

```

One way to inspect the schema of the CRD is to use the `kubectl describe` command:

```

$ kubectl describe crds mongodbcommunity.mongodbcommunity.mongodb.com

```

You will find that the output of the command is very lengthy. Looking through the details, you will see that the type is called `MongoDBCommunity`. The CRD offers a lot of properties that can be set when instantiating an object of this type. See the [documentation of the operator](#) for more information.

2. Create the object from the YAML manifest file:

```
$ kubectl apply -f backup-resource.yaml
customresourcedefinition.apiextensions.k8s.io/backups.example.com created
```

You can interact with the CRD using the following command. Make sure to spell out the full name of the CRD, `backups.example.com`:

```
$ kubectl get crd backups.example.com
NAME                                CREATED AT
backups.example.com                2023-05-24T15:11:15Z
$ kubectl describe crd backups.example.com
...
```

Create the YAML manifest in file *backup.yaml* that uses the CRD kind Backup:

```
apiVersion: example.com/v1
kind: Backup
metadata:
  name: nginx-backup
spec:
  cronExpression: "0 0 * * *"
  podName: nginx
  path: /usr/local/nginx
```

Create the object from the YAML manifest file:

```
$ kubectl apply -f backup.yaml
backup.example.com/nginx-backup created
```

You can interact with the object using the built-in `kubectl` commands for any other Kubernetes API primitive:

```
$ kubectl get backups
NAME            AGE
nginx-backup    24s
$ kubectl describe backup nginx-backup
...
```

Chapter 8, Helm and Kustomize

1. Add the Helm chart repository using the provided URL. The name assigned to the URL is `prometheus-community`:

```
$ helm repo add prometheus-community https://prometheus-community.\
github.io/helm-charts
"prometheus-community" has been added to your repositories
```

Update the chart information with the following command:

```
$ helm repo update prometheus-community
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" \
chart repository
Update Complete. *Happy Helming!*
```

You can search published chart versions in the repository named `prometheus-community`:

```
$ helm search hub prometheus-community
```

URL	CHART VERSION	..
https://artifacthub.io/packages/helm/prometheus...	70.3.0	..

Install the latest version of the chart `kube-prometheus-stack`:

```
$ helm install prometheus prometheus-community/kube-prometheus-stack
NAME: prometheus
LAST DEPLOYED: Wed Mar 26 14:14:53 2025
NAMESPACE: default
STATUS: deployed
REVISION: 1
...
```

The installed charts can be listed with the following command:

```
$ helm list
```

NAME	NAMESPACE	REVISION	UPDATED	...
prometheus	default	1	2025-03-26	...

One of the objects created by the chart is the Service named `prometheus-operated`. This Service exposes the Prometheus dashboard on port 9090:

```
$ kubectl get service prometheus-operated
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	...
prometheus-operated	ClusterIP	None	<none>	...

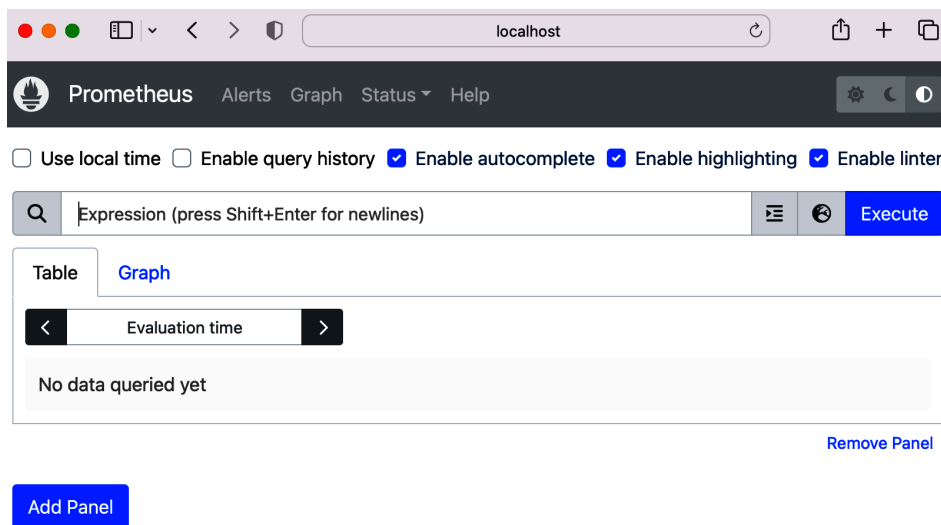
Set up port forwarding from port 8080 to port 9090:

```
$ kubectl port-forward service/prometheus-operated 8080:9090
```

```
Forwarding from 127.0.0.1:8080 -> 9090
```

```
Forwarding from [::1]:8080 -> 9090
```

Open a browser and enter the URL <http://localhost:8080/>. You will be presented with the Prometheus dashboard.



Uninstall the chart with the following command:

```
$ helm uninstall prometheus
```

```
release "prometheus" uninstalled
```

2. Navigate to the folder containing the *manifests* directory. Create all objects contained in the *manifests* directory using the recursive `apply` command:

```
$ kubectl apply -f manifests/ -R
```

```
configmap/logs-config created
```

```
pod/nginx created
```

Modify the value of the key `dir` in the file *configmap.yaml* using an editor. Then update the live object of the ConfigMap using the following command:

```
$ vim manifests/configmap.yaml
$ kubectl apply -f manifests/configmap.yaml
configmap/logs-config configured
```

Delete all objects that have been created from the *manifests* directory using the recursive `delete` command:

```
$ kubectl delete -f manifests/ -R
configmap "logs-config" deleted
pod "nginx" deleted
```

Create the file *kustomization.yaml*. It should define the common attribute for the namespace and reference the resource with the file *pod.yaml*. The following YAML file shows its contents:

```
namespace: t012
resources:
- pod.yaml
```

Run the following `kustomize` command to render the transformed manifest as console output:

```
$ kubectl kustomize ./
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: t012
spec:
  containers:
  - image: nginx:1.21.1
    name: nginx
```

Chapter 9, Pods and Namespaces

1. You can either use the imperative approach or the declarative approach. First, we'll look at creating the namespace with the imperative approach:

```
$ kubectl create namespace j43
```

Create the Pod:


```
$ kubectl run nginx --image=nginx:1.17.10 --port=80 --namespace=j43
```

Alternatively, you can use the declarative approach. Create a new YAML manifest in the file called *namespace.yaml* with the following contents:

```
apiVersion: v1
kind: Namespace
metadata:
  name: j43
```

Create the namespace from the YAML manifest:

```
$ kubectl apply -f namespace.yaml
```

Create a new YAML manifest in the file *nginx-pod.yaml* with the following contents:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.17.10
    ports:
    - containerPort: 80
```

Create the Pod from the YAML manifest:

```
$ kubectl apply -f nginx-pod.yaml --namespace=j43
```

You can use the command-line option `-o wide` to retrieve the IP address of the Pod:

```
$ kubectl get pod nginx --namespace=j43 -o wide
```

The same information is available if you query for the Pod details:

```
$ kubectl describe pod nginx --namespace=j43 | grep IP:
```

You can use the command-line options `--rm` and `-it` to start a temporary Pod. The following command assumes that the IP address of the Pod named `nginx` is 10.1.0.66:

```
$ kubectl run busybox --image=busybox:1.36.1 --restart=Never --rm -it \
-n j43 -- wget -O- 10.1.0.66:80
```

To download the logs, use a simple `logs` command:

```
$ kubectl logs nginx --namespace=j43
```

Editing the live object is forbidden. You will receive an error message if you try to add the environment variables:

```
$ kubectl edit pod nginx --namespace=j43
```

You will have to re-create the object with a modified YAML manifest, but first you'll have to delete the existing object:

```
$ kubectl delete pod nginx --namespace=j43
```

Edit the existing YAML manifest in the file *nginx-pod.yaml*:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.17.10
    ports:
    - containerPort: 80
    env:
    - name: DB_URL
      value: postgresql://mydb:5432
    - name: DB_USERNAME
      value: admin
```

Apply the changes:

```
$ kubectl apply -f nginx-pod.yaml --namespace=j43
```

Use the `exec` command to open an interactive shell to the container:

```
$ kubectl exec -it nginx --namespace=j43 -- /bin/sh
# ls -l
```

2. Combine the command-line options `-o yaml` and `--dry-run=client` to write the generated YAML to a file. Make sure to escape the double quote characters of the string rendered by the `echo` command:

```
$ kubectl run loop --image=busybox:1.36.1 -o yaml --dry-run=client \
--restart=Never -- /bin/sh -c 'for i in 1 2 3 4 5 6 7 8 9 10; \
do echo "Welcome $i times"; done' \
> pod.yaml
```

Create the Pod from the YAML manifest:

```
$ kubectl apply -f pod.yaml --namespace=j43
```

The status of the Pod will say `Completed`, as the executed command in the container does not run in an infinite loop:

```
$ kubectl get pod loop --namespace=j43
```

The container command cannot be changed for existing Pods. Delete the Pod so you can modify the manifest file and re-create the object:

```
$ kubectl delete pod loop --namespace=j43
```

Change the YAML manifest content:

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: loop
  name: loop
spec:
  containers:
  - args:
    - /bin/sh
    - -c
    - while true; do date; sleep 10; done
```

```
image: busybox:1.36.1
name: loop
resources: {}
dnsPolicy: ClusterFirst
restartPolicy: Never
status: {}
```

Create the Pod from the YAML manifest:

```
$ kubectl apply -f pod.yaml --namespace=j43
```

You can describe the Pod events by grepping for the term:

```
$ kubectl describe pod loop --namespace=j43 | grep -C 10 Events:
```

You can simply delete the namespace, which will delete all objects within the namespace:

```
$ kubectl delete namespace j43
```

Chapter 10, ConfigMaps and Secrets

1. Create the ConfigMap and point to the text file upon creation:

```
$ kubectl create configmap app-config --from-file=application.yaml
configmap/app-config created
```

The ConfigMap defines a single key-value pair. The key is the name of the YAML file, and the value is the contents of *application.yaml*:

```
$ kubectl get configmap app-config -o yaml
apiVersion: v1
data:
  application.yaml: |-
    dev:
      url: http://dev.bar.com
      name: Developer Setup
    prod:
      url: http://foo.bar.com
      name: My Cool App
kind: ConfigMap
metadata:
  creationTimestamp: "2023-05-22T17:47:52Z"
  name: app-config
```

```
namespace: default
resourceVersion: "7971"
uid: 00cf4ce2-ebec-48b5-a721-e1bde2aabd84
```

Execute the `run` command in combination with the `--dry-run` flag to generate the file for the Pod:

```
$ kubectl run backend --image=nginx:1.23.4-alpine -o yaml \
  --dry-run=client --restart=Never > pod.yaml
```

The final YAML manifest should look similar to the following code snippet:

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: backend
  name: backend
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: backend
    volumeMounts:
    - name: config-volume
      mountPath: /etc/config
  volumes:
  - name: config-volume
    configMap:
      name: app-config
```

Create the Pod by pointing the `apply` command to the YAML manifest:

```
$ kubectl apply -f pod.yaml
pod/backend created
```

Log into the Pod and navigate to the directory `/etc/config`. You will find the file `application.yaml` with the expected YAML content:

```
$ kubectl exec backend -it -- /bin/sh
/ # cd /etc/config
/etc/config # ls
application.yaml
/etc/config # cat application.yaml
dev:
```

```
    url: http://dev.bar.com
    name: Developer Setup
prod:
    url: http://foo.bar.com
    name: My Cool App
/etc/config # exit
```

2. It's easy to create the Secret from the command line:

```
$ kubectl create secret generic db-credentials --from-literal=\
db-password=password
secret/db-credentials created
```

The imperative command automatically Base64-encodes the provided value of the literal. You can render the details of the Secret object from the command line. The assigned value to the key `db-password` is `cGFzc3dk`:

```
$ kubectl get secret db-credentials -o yaml
apiVersion: v1
data:
  db-password: cGFzc3dk
kind: Secret
metadata:
  creationTimestamp: "2023-05-22T16:47:33Z"
  name: db-credentials
  namespace: default
  resourceVersion: "7557"
  uid: 2daf580a-b672-40dd-8c37-a4adb57a8c6c
type: Opaque
```

Execute the `run` command in combination with the `--dry-run` flag to generate the file for the Pod:

```
$ kubectl run backend --image=nginx:1.23.4-alpine -o yaml \
--dry-run=client --restart=Never > pod.yaml
```

Edit the YAML manifest and create an environment that reads the key from the Secret while assigning a new name for it:

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: backend
  name: backend
```

```
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: backend
    env:
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: db-password
```

Create the Pod by pointing the `apply` command to the YAML manifest:

```
$ kubectl apply -f pod.yaml
pod/backend created
```

You can find the environment variable in Base64-decoded form by shelling into the container and running the `env` command:

```
$ kubectl exec -it backend -- env
DB_PASSWORD=passwd
```

Chapter 11, Deployments and ReplicaSets

1. Execute the command to create the Deployment object. You will see an error message:

```
$ kubectl apply -f fix-me-deployment.yaml
The Deployment "nginx-deployment" is invalid: spec.template.metadata.labels: Invalid value: map[string]string{"app":"nginx"}: 'selector' does not \
match template 'labels'
```

The label selector does not match with the assigned labels in the Pod template. Change the manifest so that they line up. The manifest that follows uses `run: server` under `spec.selector.matchLabels` and `spec.template.metadata.labels`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
```

```

    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      run: server
  template:
    metadata:
      labels:
        run: server
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80

```

You should now be able to create the object:

```

$ kubectl apply -f fix-me-deployment.yaml
deployment.apps/nginx-deployment created

```

2. Create the YAML manifest for a Deployment in the file *nginx-deployment.yaml*. The label selector should match the labels assigned to the Pod template. The following code snippet shows the contents of the YAML manifest file:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    tier: backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: v1
  template:
    metadata:
      labels:
        app: v1
    spec:
      containers:
      - image: nginx:1.23.0
        name: nginx

```


Create the deployment by pointing it to the YAML manifest. Check on the Deployment status:

```
$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx created
$ kubectl get deployment nginx
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	3/3	3	3	10s

Set the new image and check the revision history:

```
$ kubectl set image deployment/nginx nginx=nginx:1.23.4
deployment.apps/nginx image updated
$ kubectl rollout history deployment nginx
deployment.apps/nginx
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
$ kubectl rollout history deployment nginx --revision=2
deployment.apps/nginx with revision #2
Pod Template:
  Labels:      app=v1
              pod-template-hash=5bd95c598
  Containers:
    nginx:
      Image:      nginx:1.23.4
      Port:      <none>
      Host Port:  <none>
      Environment:  <none>
      Mounts:      <none>
      Volumes:     <none>
```

Add the change cause to the current revision by annotating the Deployment object:

```
$ kubectl annotate deployment nginx kubernetes.io/change-cause=\
"Pick up patch version"
deployment.apps/nginx annotated
```

The revision change cause can be inspected by rendering the rollout history:

```
$ kubectl rollout history deployment nginx
deployment.apps/nginx
REVISION  CHANGE-CAUSE
```

```
1          <none>
2          Pick up patch version
```

Now, scale the Deployment to five replicas. You should find five Pods controlled by the Deployment:

```
$ kubectl scale deployment nginx --replicas=5
deployment.apps/nginx scaled
$ kubectl get pod -l app=v1
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-5bd95c598-25z4j	1/1	Running	0	3m39s
nginx-5bd95c598-46mlt	1/1	Running	0	3m38s
nginx-5bd95c598-bszvp	1/1	Running	0	48s
nginx-5bd95c598-dwr8r	1/1	Running	0	48s
nginx-5bd95c598-kjrvf	1/1	Running	0	3m37s

Roll back to revision 1. You will see the new revision. Inspecting the revision should show the image `nginx:1.23.0`:

```
$ kubectl rollout undo deployment/nginx --to-revision=1
deployment.apps/nginx rolled back
$ kubectl rollout history deployment nginx
deployment.apps/nginx
REVISION  CHANGE-CAUSE
2          Pick up patch version
3          <none>
$ kubectl rollout history deployment nginx --revision=3
deployment.apps/nginx with revision #3
Pod Template:
  Labels:      app=v1
              pod-template-hash=f48dc88cd
  Containers:
    nginx:
      Image:      nginx:1.23.0
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:      <none>
  Volumes:      <none>
```

Chapter 12, Scaling Workloads

1. Create a new YAML file named *hello-world-deployment.yaml* with the contents as shown:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
spec:
  replicas: 3
  selector:
    matchLabels:
      run: hello-world
  template:
    metadata:
      labels:
        run: hello-world
    spec:
      containers:
      - image: bmuschko/nodejs-hello-world:1.0.0
        name: hello-world

```

Create the Deployment object from the manifest file:

```

$ kubectl apply -f hello-world-deployment.yaml
deployment.apps/hello-world created

```

Ensure the correct number of replicas. You should see three replicas ready:

```

$ kubectl get deployment hello-world

```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hello-world	3/3	3	3	32s

Modify the YAML file *hello-world-deployment.yaml* by editing it with an editor. Change the value of the attribute `spec.replicas` to eight. The following command uses `vim`:

```

$ vim hello-world-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
spec:
  replicas: 8
...

```

Apply the changes to the existing object:

```
$ kubectl apply -f hello-world-deployment.yaml
deployment.apps/hello-world configured
```

Ensure the correct number of replicas. You should see eight replicas ready:

```
$ kubectl get deployment hello-world
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hello-world	8/8	8	8	56s

2. Define the Deployment in the file *nginx-deployment.yaml*, as shown:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.23.4
        name: nginx
        resources:
          requests:
            cpu: "0.5"
            memory: "500Mi"
          limits:
            memory: "500Mi"
```

Create the Deployment object from the manifest file:

```
$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx created
```

Ensure that all Pods controlled by the Deployment transition into Running status:

```
$ kubectl get deployment nginx
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
------	-------	------------	-----------	-----

```

nginx      1/1      1      1      49s
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-5bbd9746c-9b4np              1/1     Running   0          24s

```

Next, define the HorizontalPodAutoscaler with the given resource thresholds in the file *nginx-hpa.yaml*. The final manifest is shown here:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx
  minReplicas: 3
  maxReplicas: 8
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 75
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 60

```

Create the HorizontalPodAutoscaler object from the manifest file:

```

$ kubectl apply -f nginx-hpa.yaml
horizontalpodautoscaler.autoscaling/nginx-hpa created

```

When you inspect the HorizontalPodAutoscaler object, you will find that the number of replicas will be scaled up to the minimum number three even though the Deployment defines only a single replica. At the time of running this command, Pods are not using a significant amount of CPU and memory. That's why the current metrics show 0%:

```

$ kubectl get hpa nginx-hpa
NAME                                REFERENCE                                TARGETS      MINPODS   MAXPODS \
REPLICAS      AGE

```

Chapter 13, Resource Requirements, Limits, and Quotas

1. Start by creating a basic definition of a Pod. The following YAML manifest defines the Pod named `hello` with a single container running the image `bmuschko/nodejs-hello-world:1.0.0`. Add a volume of type `emptyDir` to the Pod and mount it in the container. Finally, define the resource requirements for the container:

```
apiVersion: v1
kind: Pod
metadata:
  name: hello
spec:
  containers:
  - image: bmuschko/nodejs-hello-world:1.0.0
    name: hello
    ports:
    - name: nodejs-port
      containerPort: 3000
    volumeMounts:
    - name: log-volume
      mountPath: "/var/log"
  resources:
    requests:
      cpu: 100m
      memory: 500Mi
      ephemeral-storage: 1Gi
    limits:
      memory: 500Mi
      ephemeral-storage: 2Gi
  volumes:
  - name: log-volume
    emptyDir: {}
```

Create the Pod object with the following command:

```
$ kubectl apply -f pod.yaml
pod/hello created
```

The cluster in this scenario consists of three nodes: one control plane node and two worker nodes. Be aware that your setup will likely look different:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	65s	v1.32.2
minikube-m02	Ready	<none>	44s	v1.32.2
minikube-m03	Ready	<none>	26s	v1.32.2

The `-o wide` flag renders the node that the Pod is running on, in this case the node named `minikube-m03`:

```
$ kubectl get pod hello -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
hello	1/1	Running	0	25s	10.244.2.2	minikube-m03

The details of the Pod provide information about the container's resource requirements:

```
$ kubectl describe pod hello
```

```
...
Containers:
  hello:
    ...
    Limits:
      ephemeral-storage: 2Gi
      memory: 500Mi
    Requests:
      cpu: 100m
      ephemeral-storage: 1Gi
      memory: 500M
...

```

2. First, create the namespace and the resource quota in the namespace:

```
$ kubectl create namespace rq-demo
```

```
namespace/rq-demo created
```

```
$ kubectl apply -f resourcequota.yaml --namespace=rq-demo
```

```
resourcequota/app created
```

Inspect the details of the resource quota:

```
$ kubectl describe quota app --namespace=rq-demo
```

Name:	app	
Namespace:	rq-demo	
Resource	Used	Hard
-----	----	----
Pods	0	2

```
requests.cpu      0      2
requests.memory   0      500Mi
```

Next, create the YAML manifest in the file *pod.yaml* with more requested memory than available in the quota. Start by running the command `kubectl run mypod --image=nginx -o yaml --dry-run=client --restart=Never > pod.yaml`, and then edit the produced file. Remember to replace the `resources` attribute that has been created automatically:

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - image: nginx
    name: mypod
    resources:
      requests:
        cpu: "0.5"
        memory: "1Gi"
  restartPolicy: Never
```

Create the Pod and observe the error message:

```
$ kubectl apply -f pod.yaml --namespace=rq-demo
Error from server (Forbidden): error when creating "pod.yaml": pods \
"mypod" is forbidden: exceeded quota: app, requested: \
requests.memory=1Gi, used: requests.memory=0, limited: \
requests.memory=500Mi
```

Lower the memory settings to less than 500 Mi (e.g., 255 Mi) and create the Pod:

```
$ kubectl apply -f pod.yaml --namespace=rq-demo
pod/mypod created
```

The consumed resources of the Pod can be viewed in the `Used` column:

```
$ kubectl describe quota --namespace=rq-demo
Name:          app
Namespace:     rq-demo
Resource       Used   Hard
```


-----	----	-----
pods	1	2
requests.cpu	500m	2
requests.memory	255Mi	500Mi

3. Create the objects from the given YAML manifest. The file defines a namespace and a LimitRange object:

```
$ kubectl apply -f setup.yaml
namespace/d92 created
limitrange/cpu-limit-range created
```

Describing the LimitRange object gives away its container configuration details:

```
$ kubectl describe limitrange cpu-limit-range -n d92
Name:          cpu-limit-range
Namespace:     d92
Type           Resource  Min    Max    Default Request  Default Limit  ...
----           -
Container      cpu       200m   500m   500m              500m            ...
```

Define a Pod in the file *pod-without-resource-requirements.yaml* without any resource requirements:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-without-resource-requirements
  namespace: d92
spec:
  containers:
    - image: nginx:1.23.4-alpine
      name: nginx
```

Create the Pod object using the `apply` command:

```
$ kubectl apply -f pod-without-resource-requirements.yaml
pod/pod-without-resource-requirements created
```

A Pod without specified resource requirements will use the default request and limit defined by LimitRange, in this case 500 m:

```
$ kubectl describe pod pod-without-resource-requirements -n d92
...
Containers:
  nginx:
    Limits:
      cpu: 500m
    Requests:
      cpu: 500m
```

The Pod defined in the file *pod-with-more-cpu-resource-requirements.yaml* specifies a higher CPU resource limit than allowed by the LimitRange:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-more-cpu-resource-requirements
  namespace: d92
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: nginx
    resources:
      requests:
        cpu: 400m
      limits:
        cpu: 1.5
```

As a result, the Pod will not be allowed to be scheduled:

```
$ kubectl apply -f pod-with-more-cpu-resource-requirements.yaml
Error from server (Forbidden): error when creating \
"pod-with-more-cpu-resource-requirements.yaml": pods \
"pod-with-more-cpu-resource-requirements" is forbidden: \
maximum cpu usage per Container is 500m, but limit is 1500m
```

Finally, define a Pod in the file *pod-with-less-cpu-resource-requirements.yaml*. The CPU resource request and limit fits within the boundaries of the LimitRange:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-less-cpu-resource-requirements
  namespace: d92
spec:
  containers:
```

```

- image: nginx:1.23.4-alpine
  name: nginx
  resources:
    requests:
      cpu: 350m
    limits:
      cpu: 400m

```

Create the Pod object using the `apply` command:

```

$ kubectl apply -f pod-with-less-cpu-resource-requirements.yaml
pod/pod-with-less-cpu-resource-requirements created

```

The Pod uses the provided CPU resource request and limit:

```

$ kubectl describe pod pod-with-less-cpu-resource-requirements -n d92
...
Containers:
  nginx:
    Limits:
      cpu: 400m
    Requests:
      cpu: 350m

```

Chapter 14, Pod Scheduling

1. The following solution demonstrates the use of a multi-node minikube cluster. Listing the nodes renders the following output:

```

$ kubectl get nodes

```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	3m16s	v1.32.2
minikube-m02	Ready	<none>	3m6s	v1.32.2
minikube-m03	Ready	<none>	2m59s	v1.32.2

Let's assign the label `color=green` to the node `minikube-m02` and the label `color=red` to the node `minikube-m03`:

```

$ kubectl label nodes minikube-m02 color=green
node/minikube-m02 labeled
$ kubectl label nodes minikube-m03 color=red
node/minikube-m03 labeled

```

You can render the assign labels for all nodes using the `--show-labels` command line option:

```
$ kubectl get nodes --show-labels
```

NAME	STATUS	ROLES	AGE	VERSION	LABELS
minikube	Ready	control-plane	27m	v1.32.2	...
minikube-m02	Ready	<none>	27m	v1.32.2	...,color=green
minikube-m03	Ready	<none>	26m	v1.32.2	...,color=red,.

The Pod manifest could look as such:

```
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  nodeSelector:
    color: green
  containers:
  - name: nginx
    image: nginx:1.27.1
```

Create the Pod and inspect the assigned node:

```
$ kubectl apply -f pod.yaml
pod/app created
$ kubectl get pod app -o=wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
app	1/1	Running	0	21s	10.244.1.2	minikube-m02

Change the Pod manifest to use node affinity to schedule it on node `minikube-m02` or `minikube-m03`. The resulting YAML definition would look like this:

```
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: color
            operator: In
            values:
```

```

- green
- red
containers:
- name: nginx
  image: nginx:1.27.1

```

First delete the Pod, then recreate it. The Pod should be scheduled on one of the nodes:

```

$ kubectl delete -f pod.yaml
pod "app" deleted
$ kubectl apply -f pod.yaml
pod/app created
$ kubectl get pod app -o=wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
app	1/1	Running	0	12s	10.244.1.3	minikube-m02

2. The following solution demonstrates the use of a multi-node minikube cluster. Listing the nodes renders the following output:

```

$ kubectl get nodes

```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	3m16s	v1.32.2
minikube-m02	Ready	<none>	3m6s	v1.32.2
minikube-m03	Ready	<none>	2m59s	v1.32.2

The Pod definition in the file *pod.yaml* could look as follows:

```

apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  containers:
  - name: nginx
    image: nginx:1.27.1

```

Create the Pod and inspect the assigned node:

```

$ kubectl apply -f pod.yaml
pod/app created
$ kubectl get pod app -o=wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
app	1/1	Running	0	89s	10.244.2.2	minikube-m03

Add the taint to the node. In this case, the node is called minikube-m03 :

```
$ kubectl taint nodes minikube-m03 exclusive=yes:NoExecute
node/minikube-m03 tainted
```

The taint causes the Pod to be evicted:

```
$ kubectl get pods
No resources found in default namespace.
```

Change the Pod manifest and add the toleration:

```
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  tolerations:
  - key: "exclusive"
    operator: "Equal"
    value: "yes"
    effect: "NoExecute"
  containers:
  - name: nginx
    image: nginx:1.27.1
```

With the toleration, the Pod will be allowed to be scheduled on the node minikube-m03 again:

```
$ kubectl apply -f pod.yaml
pod/app created
$ kubectl get pod app -o=wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	
app	1/1	Running	0	9s	10.244.2.3	minikube-m03	.

Remove the taint from the node. The Pod will continue to run on the node:

```
$ kubectl taint nodes minikube-m03 exclusive-
node/minikube-m03 untainted
$ kubectl get pod app -o=wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	
app	1/1	Running	0	37s	10.244.2.3	minikube-m03	.

Chapter 15, Volumes

1. Start by generating the YAML manifest using the `run` command in combination with the `--dry-run` option:

```
$ kubectl run alpine --image=alpine:3.22.2 --dry-run=client \
  --restart=Never -o yaml -- /bin/sh -c "while true; do sleep 60; \
  done;" > multi-container-alpine.yaml
$ vim multi-container-alpine.yaml
```

After editing the Pod, the manifest could look like the following. The container names here are `container1` and `container2`:

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: alpine
  name: alpine
spec:
  containers:
  - args:
    - /bin/sh
    - -c
    - while true; do sleep 60; done;
    image: alpine:3.22.2
    name: container1
    resources: {}
  - args:
    - /bin/sh
    - -c
    - while true; do sleep 60; done;
    image: alpine:3.12.0
    name: container2
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

Edit the YAML manifest further by adding the volume and the mount paths for both containers.

In the end, the Pod definition could look like this:

```
apiVersion: v1
kind: Pod
metadata:
```

```

creationTimestamp: null
labels:
  run: alpine
name: alpine
spec:
  volumes:
  - name: shared-vol
    emptyDir: {}
  containers:
  - args:
    - /bin/sh
    - -c
    - while true; do sleep 60; done;
    image: alpine:3.12.0
    name: container1
    volumeMounts:
    - name: shared-vol
      mountPath: /etc/a
    resources: {}
  - args:
    - /bin/sh
    - -c
    - while true; do sleep 60; done;
    image: alpine:3.12.0
    name: container2
    volumeMounts:
    - name: shared-vol
      mountPath: /etc/b
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}

```

Create the Pod and check if it has been created properly. You should see the Pod in `Running` status with two containers ready:

```

$ kubectl apply -f multi-container-alpine.yaml
pod/alpine created
$ kubectl get pods

```

NAME	READY	STATUS	RESTARTS	AGE
alpine	2/2	Running	0	18s

Use the `exec` command to shell into the container named `container1`. Create the file `/etc/a/data/hello.txt` with the relevant content:

```

$ kubectl exec alpine -c container1 -it -- /bin/sh
/ # cd /etc/a
/etc/a # ls -l

```



```
total 0
/etc/a # mkdir data
/etc/a # cd data/
/etc/a/data # echo "Hello World" > hello.txt
/etc/a/data # cat hello.txt
Hello World
/etc/a/data # exit
```

Use the `exec` command to shell into the container named `container2`. The contents of the file `/etc/b/data/hello.txt` should say `Hello World`:

```
$ kubectl exec alpine -c container2 -it -- /bin/sh
/ # cat /etc/b/data/hello.txt
Hello World
/ # exit
```

Chapter 16, Persistent Volumes

1. Start by creating a new file named `logs-pv.yaml`. The contents could look as follows:

```
kind: PersistentVolume
apiVersion: v1
metadata:
  name: logs-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
    - ReadOnlyMany
  hostPath:
    path: /var/logs
```

Create the `PersistentVolume` object and check its status:

```
$ kubectl apply -f logs-pv.yaml
persistentvolume/logs-pv created
$ kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM \
	STORAGECLASS	REASON	AGE		
logs-pv	5Gi	RWO,ROX	Retain	Available	\

18s

Create the file *logs-pvc.yaml* to define the PersistentVolumeClaim. The following YAML manifest shows its contents:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: logs-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
  storageClassName: ""
```

Create the PersistentVolume object and check on its status:

```
$ kubectl apply -f logs-pvc.yaml
persistentvolumeclaim/logs-pvc created
$ kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
logs-pvc	Bound	logs-pv	5Gi	RWO,ROX	

Create the basic YAML manifest using the `--dry-run` command-line option:

```
$ kubectl run nginx --image=nginx:1.25.1 --dry-run=client \
-o yaml > nginx-pod.yaml
```

Now, edit the file *nginx-pod.yaml* and bind the PersistentVolumeClaim to it:

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  volumes:
    - name: logs-volume
      persistentVolumeClaim:
        claimName: logs-pvc
  containers:
    - image: nginx:1.25.1
      name: nginx
```

```

    volumeMounts:
      - mountPath: "/var/log/nginx"
        name: logs-volume
    resources: {}
    dnsPolicy: ClusterFirst
    restartPolicy: Never
  status: {}

```

Create the Pod using the following command and check its status:

```

$ kubectl apply -f nginx-pod.yaml
pod/nginx created
$ kubectl get pods

```

NAME	READY	STATUS	RESTARTS	AGE
nginx	1/1	Running	0	8s

Use the `exec` command to open an interactive shell to the Pod and create a file in the mounted directory:

```

$ kubectl exec nginx -it -- /bin/sh
# cd /var/log/nginx
# touch my-nginx.log
# ls
access.log  error.log  my-nginx.log
# exit

```

After you re-create the Pod, the file stored on the PersistentVolume should still exist:

```

$ kubectl delete pod nginx
pod "nginx" deleted
$ kubectl apply -f nginx-pod.yaml
pod/nginx created
$ kubectl exec nginx -it -- /bin/sh
# cd /var/log/nginx
# ls
access.log  error.log  my-nginx.log
# exit

```

2. Create the file `db-pvc.yaml` to define the PersistentVolumeClaim. The following YAML manifest shows its contents:

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-pvc

```

```

    namespace: persistence
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: local-path
resources:
  requests:
    storage: 128Mi

```

Create the PersistentVolumeClaim object from the YAML manifest:

```

$ kubectl apply -f db-pvc.yaml
persistentvolumeclaim/db-pvc created

```

Check the status of the PersistentVolumeClaim object:

```

$ kubectl get pvc -n persistence

```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	..
db-pvc	Pending				local-path	..

The corresponding PersistentVolume object should not have been created yet:

```

$ kubectl get pv -n persistence
No resources found

```

Create the basic YAML manifest using the `--dry-run` command-line option:

```

$ kubectl run app-consuming-pvc --image=alpine:3.21.3 -n persistence \
  --dry-run=client --restart=Never -o yaml \
  -- /bin/sh -c "while true; do sleep 60; done;" \
  > alpine-pod.yaml

```

Now, edit the file *alpine-pod.yaml* and bind the PersistentVolumeClaim to it:

```

apiVersion: v1
kind: Pod
metadata:
  name: app-consuming-pvc
  namespace: persistence
spec:
  volumes:
    - name: app-storage

```

```

    persistentVolumeClaim:
      claimName: db-pvc
  containers:
  - image: alpine:3.21.3
    name: app
    command: ["/bin/sh"]
    args: ["-c", "while true; do sleep 60; done;"]
    volumeMounts:
    - mountPath: "/mnt/data"
      name: app-storage
  restartPolicy: Never

```

Create the Pod using the following command:

```

$ kubectl apply -f alpine-pod.yaml
pod/app-consuming-pvc created

```

Ensure that the Pod transitions into the `Running` status:

```

$ kubectl get pods -n persistence

```

NAME	READY	STATUS	RESTARTS	AGE
app-consuming-pvc	1/1	Running	0	8s

The `PersistentVolume` object has been provisioned dynamically:

```

$ kubectl get pv -n persistence

```

NAME	CAPACITY	ACCESS MODES	..
pvc-af39068d-0cc2-4625-8a56-7b5207b79ace	128Mi	RWO	..

Use the `exec` command to open an interactive shell to the Pod, and create a file in the mounted directory:

```

$ kubectl exec app-consuming-pvc -n persistence -it -- /bin/sh
# cd /mnt/data
# touch test.db
# ls
test.db
# exit

```

The file will be persisted in the `PersistentVolume`.

Chapter 17, Services

1. Create the Deployment definition in the file *deployment.yaml*. Be sure to expose two container ports, 80 and 9090:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
        - name: webapp
          image: nginxdemos/hello:0.4-plain-text
          ports:
            - containerPort: 80
            - containerPort: 9090
```

Define the Service in the file *service.yaml*. The most important part is the port mapping. Assign the static node port 30080 only to the web port mapping:

```
apiVersion: v1
kind: Service
metadata:
  name: webapp-service
spec:
  type: NodePort
  selector:
    app: webapp
  ports:
    - name: web
      port: 80
      targetPort: 80
      nodePort: 30080
    - name: metrics
      port: 9090
      targetPort: 9090
```

Create the application Deployment and Service objects:

```
$ kubectl apply -f deployment.yaml
$ kubectl apply -f service.yaml
```

When listed, the Deployment and Service objects are going to look similar:

```
$ kubectl get deployment,service
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/webapp	3/3	3	3	6s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	AGE
service/webapp-service	NodePort	10.111.80.190	<none>	6s
80:30080/TCP,9090:31231/TCP				

To test accessibility from your host machine, you can configure port forwarding for the Service. Here, we are mapping the host port 9091 to the Service port 80:

```
$ kubectl port-forward service/webapp-service 9091:80 &
Forwarding from 127.0.0.1:9091 -> 80
Forwarding from [::1]:9091 -> 80
Handling connection for 9091
```

You can now access the Service from your host machine using `curl` :

```
$ curl localhost:9091
Server address: 127.0.0.1:80
Server name: webapp-6dc64898b-hw116
Date: 21/Aug/2025:02:09:01 +0000
URI: /
Request ID: b5e0f6324ad7bac1b513dba9d2c1cf64
```

2. Create the database Deployment definition in the file *database-deployment.yaml*. Set the environment variables and expose the container port 3306:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: database
spec:
  replicas: 1
  selector:
    matchLabels:
```

```

    app: database
  template:
    metadata:
      labels:
        app: database
    spec:
      containers:
        - name: mysql
          image: mysql:9.4.0
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: secretpass
            - name: MYSQL_DATABASE
              value: myapp
          ports:
            - containerPort: 3306

```

Define the database Service in the file *database-service.yaml*. Map the port 3306 to the target port 3306:

```

apiVersion: v1
kind: Service
metadata:
  name: database-service
spec:
  type: ClusterIP
  selector:
    app: database
  ports:
    - port: 3306
      targetPort: 3306

```

Define the frontend Deployment in the file *frontend-deployment.yaml*. Be sure to define the command in the correct format:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:

```



```
containers:
- name: frontend
  image: busybox:1.35
  command:
  - sh
  - -c
  - "while true; do nc -zv database-service 3306; sleep 5; done"
```

Create the application Deployment and Service objects:

```
$ kubectl apply -f database-deployment.yaml
$ kubectl apply -f database-service.yaml
$ kubectl apply -f frontend-deployment.yaml
```

The frontend Pods can be selected by the label `app=frontend`. The logs of both Pods should indicate that the connection to the database Service could be established. An error message in the logs indicates a configuration issue:

```
$ kubectl logs -l app=frontend
database-service (10.101.125.103:3306) open
database-service (10.101.125.103:3306) open
```

Chapter 18, Ingresses

1. Define the namespace in the file *namespace.yaml*:

```
apiVersion: v1
kind: Namespace
metadata:
  name: webapp
```

The file *deployment.yaml* shows the `frontend` and `api` Deployments:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: webapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
```

```

    template:
      metadata:
        labels:
          app: frontend
      spec:
        containers:
        - name: frontend
          image: nginx:1.29.1-alpine
          ports:
            - containerPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: webapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
      - name: api
        image: httpd:2.4.65-alpine
        ports:
          - containerPort: 80

```

The file *services.yaml* defines the frontend and api Services:

```

apiVersion: v1
kind: Service
metadata:
  name: frontend-service
  namespace: webapp
spec:
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
  type: ClusterIP
---
apiVersion: v1
kind: Service
metadata:

```

```

    name: api-service
    namespace: webapp
spec:
  selector:
    app: api
  ports:
  - port: 80
    targetPort: 80
    protocol: TCP
  type: ClusterIP

```

Lastly, define the Ingress in the file *ingress.yaml*:

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webapp-ingress
  namespace: webapp
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
      - path: /app
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80

```

Create all objects from the YAML manifests:

```
$ kubectl apply -f namespace.yaml
$ kubectl apply -f deployments.yaml
$ kubectl apply -f services.yaml
$ kubectl apply -f ingress.yaml
```

The instruction to map `app.example.com` to the Ingress hostname (typically done by editing `/etc/hosts`) is only necessary for local testing on your own machine when you want to test hostname-based routing. During the exam, this step is not required:

```
sudo vim /etc/hosts
127.0.0.1          app.example.com
```

Inspect the Ingress object. Make sure that the value in the column `ADDRESS` has been populated:

```
$ kubectl get ingress webapp-ingress -n webapp
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
webapp-ingress	nginx	app.example.com	192.168.49.2	80	2m

Test all Ingress endpoints by sending a `curl` request:

```
$ curl -H "Host: app.example.com" http://localhost/
$ curl -H "Host: app.example.com" http://localhost/app
$ curl -H "Host: app.example.com" http://localhost/api
```

2. We'll use imperative commands to set up the backend objects to speed up the process. First, we'll need to create the namespace `production-apps`:

```
$ kubectl create namespace production-apps
```

Then we'll create the blue and green Deployment objects:

```
$ kubectl create deployment app-blue \
  --image=nginxdemos/hello:0.3-plain-text \
  --replicas=3 -n production-apps
$ kubectl create deployment app-green \
  --image=nginxdemos/hello:0.4-plain-text \
  --replicas=3 -n production-apps
```

Next up, create the Service objects using the `expose deployment` command:

```
$ kubectl expose deployment app-blue --name=app-blue-svc \
  --port=80 -n production-apps
$ kubectl expose deployment app-green --name=app-green-svc \
  --port=80 -n production-apps
```

Define the main Ingress in the file *blue-ingress.yaml*:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-main
  namespace: production-apps
spec:
  ingressClassName: nginx
  rules:
  - host: app.production.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app-blue-svc
            port:
              number: 80
```

Define the canary Ingress in the file *green-ingress.yaml*:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-canary-weight
  namespace: production-apps
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "20"
spec:
  ingressClassName: nginx
  rules:
  - host: app.production.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
```

```
name: app-green-svc
port:
  number: 80
```

Create the Ingress objects from the YAML manifests:

```
$ kubectl apply -f blue-ingress.yaml
$ kubectl apply -f green-ingress.yaml
```

You can map `localhost` to the hostname used by the Ingress objects to make them accessible for local machine:

```
sudo vim /etc/hosts
127.0.0.1      app.production.com
```

Inspect the Ingress objects. Make sure that the value in the column `ADDRESS` has been populated:

```
$ kubectl get ingresses -n production-apps
```

NAME	CLASS	HOSTS	ADDRESS	PORTS
app-canary-weight	nginx	app.production.com	192.168.49.2	80
app-main	nginx	app.production.com	192.168.49.2	80

To see the traffic routing distribution in action, run a `curl` command for loop. You will see that most requests are routed to the “blue” Pod, and a subset of the requests are routed to the “green” Pod:

```
for i in {1..10}; do curl -s -H "Host: app.production.com" \
  http://localhost | grep -o "Server name:.*"; done
```

Chapter 19, Gateway API

1. You can find the installation instructions for the NGINX Gateway Fabric controller in the [official documentation](#). Follow the instructions of your preferred installation method.
Create the application Deployment and Service objects. You can use the file `setup.yaml` to set them up at once. The solution provided in this file creates content on the container mount path at `/usr/share/nginx/html/web` for the `web-app` Pods, and content on the container mount path at `/usr/local/apache2/htdocs/api` for the `api-app` Pods:

```
$ kubectl apply -f setup.yaml
```

Ensure that the objects are available:

```
$ kubectl get deployments,services,pods
```

Configure Gateway API resources. Define the Gateway in the file *gateway.yaml*:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: main-gateway
spec:
  gatewayClassName: nginx
  listeners:
  - name: http
    port: 80
    protocol: HTTP
    hostname: example.local
```

Define the Gateway in the file *httproute.yaml*:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-routes
spec:
  parentRefs:
  - name: main-gateway
  hostnames:
  - example.local
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /web
    backendRefs:
    - name: web-app
      port: 80
  - matches:
    - path:
        type: PathPrefix
        value: /api
    backendRefs:
    - name: api-app
      port: 80
```

Create the Gateway objects with the following commands:

```
$ kubectl apply -f gateway.yaml
$ kubectl apply -f httproute.yaml
```

Check whether the Gateway received the assignment of an IP address.

This may take a few minutes:

```
$ kubectl get gateway main-gateway -o \
  jsonpath='{.status.addresses[0].value}'
```

The following command verifies that the status for the `Accepted` condition of the HTTPRoute renders `True`:

```
$ kubectl get httproute app-routes -o \
  jsonpath='{.status.parents[*].conditions[?(@.type=="Accepted")].status}'
```

Port forward to the Gateway:

```
$ kubectl port-forward svc/main-gateway-nginx 8080:80 &
```

You should be able to access the Gateway using `curl`:

```
$ curl -H "Host: example.local" http://localhost:8080/web/
$ curl -H "Host: example.local" http://localhost:8080/api/
```

2. Create the namespaces and the applications running in them from the file *setup.yaml*:

```
$ kubectl apply -f setup.yaml
```

Configure the Gateway in the file *gateway.yaml*. Allow that routing is accepted from all namespaces:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: gateway
  namespace: production
spec:
  gatewayClassName: nginx
  listeners:
```


- name: http
 - port: 80
 - protocol: HTTP
 - hostname: example.com
 - allowedRoutes:
 - namespaces:
 - from: All

Create the HTTPRoute in the `production` namespace in the file *production-route.yaml*:

```

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: prod-route
  namespace: production
spec:
  parentRefs:
  - name: gateway
  hostnames:
  - example.com
  rules:
  - matches:
    - path:
        type: Exact
        value: /app
    backendRefs:
    - name: prod-web
      port: 80
    filters:
    - type: URLRewrite
      urlRewrite:
        path:
          type: ReplaceFullPath
          replaceFullPath: /
    - type: RequestHeaderModifier
      requestHeaderModifier:
        set:
        - name: X-Environment
          value: production

```

Create the HTTPRoute in the `staging` namespace in the file *staging-route.yaml*:

```

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: staging-route
  namespace: staging

```

```
spec:
  parentRefs:
  - name: gateway
    namespace: production
  hostnames:
  - example.com
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /staging
  backendRefs:
  - name: staging-web
    port: 80
  filters:
  - type: URLRewrite
    urlRewrite:
      path:
        type: ReplacePrefixMatch
        replacePrefixMatch: /
  - type: RequestHeaderModifier
    requestHeaderModifier:
      set:
      - name: X-Environment
        value: staging
```

Create the ReferenceGrant for cross-namespace access in the *reference-grant.yaml* file:

```
apiVersion: gateway.networking.k8s.io/v1beta1
kind: ReferenceGrant
metadata:
  name: allow-staging-to-gateway
  namespace: production
spec:
  from:
  - group: gateway.networking.k8s.io
    kind: HTTPRoute
    namespace: staging
  to:
  - group: gateway.networking.k8s.io
    kind: Gateway
    name: gateway
```

Create all HTTPRoute objects:

```
$ kubectl apply -f gateway.yaml
$ kubectl apply -f production-route.yaml
```

```
$ kubectl apply -f staging-route.yaml
$ kubectl apply -f reference-grant.yaml
```

Port forward to the Gateway:

```
$ kubectl port-forward -n production svc/gateway-nginx 8080:80 &
```

You should be able to access the Gateway using `curl`:

```
$ curl -H "Host: example.com" http://localhost:8080/app
$ curl -H "Host: example.com" http://localhost:8080/staging
```

Chapter 20, Network Policies

1. Create the NetworkPolicy for the Pods with the label assignment

`app=alpha-app` in the file *alpha-app-policy.yaml*. The NetworkPolicy allows egress traffic to Pods in the namespace with the assigned label `team=beta`. It denies all other egress traffic except DNS signified by the port 53:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: alpha-app-policy
  namespace: team-alpha
spec:
  podSelector:
    matchLabels:
      app: alpha-app
  policyTypes:
  - Egress
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          team: beta
  - to:
    ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
```

The NetworkPolicy defined in the file *beta-app-policy.yaml* looks a little simpler. It allows the `beta-app` Pod to receive traffic from the `team-`

alpha namespace on port 80:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: beta-app-policy
  namespace: team-beta
spec:
  podSelector:
    matchLabels:
      app: beta-app
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          team: alpha
    ports:
    - protocol: TCP
      port: 80
```

Create both NetworkPolicy objects:

```
$ kubectl apply -f alpha-app-policy.yaml
$ kubectl apply -f beta-app-policy.yaml
```

Use the `curl` command to test the `alpha-app` Pod to make a call to the `beta-app` Service in the `team-beta` namespace. The command should succeed:

```
$ kubectl exec -it alpha-app -n team-alpha -- \
  curl -v --connect-timeout 2 \
  http://beta-app.team-beta.svc.cluster.local:8080
```

Use the `curl` command to test the `alpha-app` Pod to an address on the internet. The command should fail:

```
$ kubectl exec -it alpha-app -n team-alpha -- \
  curl -v --connect-timeout 2 http://google.com
```

Use the `curl` command to test the connectivity to the `beta-app` Service from a temporary Pod in the same namespace. The command should fail:

```
$ kubectl run test-pod --image=alpine/curl:8.14.1 -n team-beta --rm -it
--restart=Never -- curl -v --connect-timeout 2 http://beta-app:8080
```

2. Network policies are additive, which means you can apply them in any order. To make the changes more understandable, you may want to start with the deny policy first. The file *default-deny-all.yaml* shows the contents of the NetworkPolicy that denies all ingress communication within the `production` namespace:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: production
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

Create the NetworkPolicy for the database component in the file *database-policy.yaml*. The NetworkPolicy applies to the Pods with the label assignment `tier=database` and only allows ingress traffic from Pods with the label assignment `tier=backend` on port 6379:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: database-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      tier: database
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              tier: backend
      ports:
        - protocol: TCP
          port: 6379
```

Create the NetworkPolicy for the backend component in the file *backend-policy.yaml*. This policy applies to Pods with the label assignment

tier=backend. It allows incoming traffic from the frontend microservice and outgoing traffic to the database microservice:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      tier: backend
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          tier: frontend
    ports:
    - protocol: TCP
      port: 80
  egress:
  - to:
    - podSelector:
        matchLabels:
          tier: database
    ports:
    - protocol: TCP
      port: 6379
  - to:
    ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
```

Create all NetworkPolicy objects:

```
$ kubectl apply -f default-deny-all.yaml
$ kubectl apply -f database-policy.yaml
$ kubectl apply -f backend-policy.yaml
```

Use the `curl` command from the `frontend` Pod to test connectivity to the `database` Pod. We are using the `telnet` protocol here to ensure it can connect to the database port. The command should fail:

```
$ kubectl exec -it frontend -n production -- curl -v --connect-timeout
telnet://database:3306
```

Use the `curl` command from the `frontend` Pod to test connectivity to the `backend` Pod. The command should succeed:

```
$ kubectl exec -it frontend -n production -- curl -v --connect-timeout
http://backend:80
```

Use the `curl` command from the `backend` Pod to test connectivity to the `database` Pod. The command should succeed. Similar to the first command, we are using the `telnet` protocol:

```
$ kubectl exec -it backend -n production -- curl -v --connect-timeout 2 \
telnet://database:3306
```

Chapter 21, Troubleshooting Applications

1. First, create the Pod using the given YAML content:

```
$ kubectl apply -f setup.yaml
pod/date-recorder created
```

Inspecting the Pod's status exposes no obvious issues. The status is `Running`:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
date-recorder	1/1	Running	0	5s

Render the logs of the container. The returned error message indicates that the file or directory `/root/tmp/startup-marker.txt` does not exist:

```
$ kubectl logs date-recorder
[Error: ENOENT: no such file or directory, open \
'/root/tmp/startup-marker.txt'] {
  errno: -2,
  code: 'ENOENT',
  syscall: 'open',
```

```
    path: '/root/tmp/curr-date.txt'
  }
}
```

We could try to open a shell to the container; however, the container image does not provide a shell:

```
$ kubectl exec -it date-recorder -- /bin/sh
OCI runtime exec failed: exec failed: unable to start container \
process: exec: "/bin/sh": stat /bin/sh: no such file or \
directory: unknown
command terminated with exit code 126
```

We can use the `debug` command to create a debugging container for troubleshooting. The `--share-processes` flag lets us share the running nodejs process:

```
$ kubectl debug -it date-recorder --image=busybox --target=debian \
--share-processes
Targeting container "debian". If you don't see processes from this \
container it may be because the container runtime doesn't support \
this feature.
Defaulting debug container name to debugger-rns89.
If you don't see a command prompt, try pressing enter.
/ # ps
PID    USER      TIME  COMMAND
   1   root         4:21 /nodejs/bin/node -e const fs = require('fs'); \
    let timestamp = Date.now(); fs.writeFile('/root/tmp/startup-m
  35   root         0:00 sh
   41   root         0:00 ps
```

Apparently, the directory we want to write to does indeed not exist:

```
$ kubectl exec failing-pod -it -- /bin/sh
/ # ls /root/tmp
ls: /root/tmp: No such file or directory
```

We'll likely want to change the command running the original container to point to the directory that does exist upon container start. Alternatively, it may make sense to mount an ephemeral volume to provide the directory, as shown here:

```
apiVersion: v1
kind: Pod
metadata:
  name: date-recorder
spec:
```



```

containers:
- name: debian
  image: gcr.io/distroless/nodejs20-debian11
  command: ["/nodejs/bin/node", "-e", "const fs = require('fs'); \
let timestamp = Date.now(); fs.writeFile('/var/startup/\
startup-marker.txt', timestamp.toString(), err => { if (err) { \
console.error(err); } while(true) {} });"]
  volumeMounts:
- mountPath: /var/startup
  name: init-volume
volumes:
- name: init-volume
  emptyDir: {}

```

2. Create the objects from the *setup.yaml* file. You will see from the output that at least three objects have been created: a namespace, a Deployment, and a Service:

```

$ kubectl apply -f setup.yaml
namespace/y72 created
deployment.apps/web-app created
service/web-app created

```

You can list all objects relevant to the scenario using the following command:

```

$ kubectl get all -n y72

```

NAME	READY	STATUS	RESTARTS	AGE
pod/web-app-5f77f59c78-8svdm	1/1	Running	0	10m
pod/web-app-5f77f59c78-mhvjz	1/1	Running	0	10m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/web-app	ClusterIP	10.106.215.153	<none>	80/TCP

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/web-app	2/2	2	2	10m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/web-app-5f77f59c78	2	2	2	10m

The Service is named `web-app` and is of type `ClusterIP`. You can access the Service only from within the cluster. Trying to connect to the Service by its DNS name from a temporary Pod in the same namespace won't be allowed:

```

$ kubectl run tmp --image=busybox --restart=Never -it --rm -n y72 \
-- wget web-app

```

```
Connecting to web-app (10.106.215.153:80)
wget: can't connect to remote host (10.106.215.153): Connection refused
pod "tmp" deleted
pod y72/tmp terminated (Error)
```

The endpoint for the Service `web-app` cannot be resolved, as shown by the following command:

```
$ kubectl get endpoints -n y72
```

NAME	ENDPOINTS	AGE
web-app	<none>	15m

Describing the Service object provides you with additional information, e.g., the label selector and the target port:

```
$ kubectl describe service web-app -n y72
```

```
Name:                web-app
Namespace:           y72
Labels:              <none>
Annotations:         <none>
Selector:            run=myapp
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.106.215.153
IPs:                 10.106.215.153
Port:                <unset> 80/TCP
TargetPort:          3001/TCP
Endpoints:           <none>
Session Affinity:    None
Events:              <none>
```

Upon inspecting the Deployment, you will find that the Pod template uses the label assignment `app=webapp`. The container port is set to 3000. This information doesn't match the configuration of the Service. The endpoints of the `web-app` Service now point to the IP address and container port of the replicas controlled by the Deployment:

```
$ kubectl get endpoints -n y72
```

NAME	ENDPOINTS	AGE
web-app	10.244.0.3:3000,10.244.0.4:3000	24m

Edit the live object of the Service. Change the label selector from `run=myapp` to `app=webapp`, and the target port from 3001 to 3000:

```
$ kubectl edit service web-app -n y72
service/web-app edited
```

After changing the Service configuration, you will find that you can open a connection to the Pod running the application:

```
$ kubectl run tmp --image=busybox:1.36.1 --restart=Never -it --rm -n y72
-- wget web-app
Connecting to web-app (10.106.215.153:80)
saving to 'index.html'
index.html          100% |*****| ...
'index.html' saved
pod "tmp" deleted
```

3. Create the namespace with:

```
$ kubectl create ns stress-test
namespace/stress-test created
```

Create all Pods by pointing the `apply` command to the current directory:

```
$ kubectl apply -f ./
pod/stress-1 created
pod/stress-2 created
pod/stress-3 created
```

Retrieve the metrics for the Pods from the Metrics Server using the `top` command:

```
$ kubectl top pods -n stress-test
NAME          CPU(cores)   MEMORY(bytes)
stress-1      50m          77Mi
stress-2      74m          138Mi
stress-3      58m          94Mi
```

The Pod with the highest amount of memory consumption is the Pod named `stress-2`. The metrics will look different on your machine given that the amount of consumed memory is randomized by the command executed per container.

Chapter 22, Troubleshooting Clusters

1. List all nodes in the cluster. You will see that the node `worker-2` shows the status `NotReady,SchedulingDisabled`:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VER
control-plane	Ready	control-plane	23m	v1.
worker-1	Ready	<none>	23m	v1.
worker-2	NotReady,SchedulingDisabled	<none>	23m	v1.

Check why scheduling is disabled. The command renders a taint that indicates that workload cannot be scheduled on the node:

```
$ kubectl describe node worker-node-2 | grep -i taint
```

Check the node for any other conditions. You will see the message “Kubelet stopped posting node status”:

```
$ kubectl describe node worker-node-2 | grep -A10 Conditions
```

Uncordon the node to allow scheduling on the node again:

```
$ kubectl uncordon worker-node-2
```

Check if the kubelet is healthy on the worker node. SSH into the node and check kubelet status. You will see that the kubelet process is inactive:

```
$ ssh worker-node-2  
$ sudo systemctl status kubelet
```

Restart the kubelet process and verify that it is running properly:

```
$ sudo systemctl restart kubelet  
$ sudo systemctl status kubelet
```

Check the node status. All nodes should show `Ready` without `SchedulingDisabled`:

```
$ kubectl get nodes
```

Run a test Pod specifically on `worker-node-2`. Verify that the Pod is running on the correct node:

```
$ kubectl run test-pod --image=nginx:1.29.1 --overrides=\
'{"spec":{"nodeSelector":{"kubernetes.io/hostname":"worker-node-2"}}}'
$ kubectl get pod test-pod -o wide
```

2. Create the Deployment with the parameters provided in the instructions:

```
$ kubectl create deployment test-app --image=nginx:1.29.1 --replicas=3
```

Check the status of the Pods controlled by the ReplicaSet. All Pods will have the status `Pending`:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
test-app-5d4d5b6c7b-h2x4m	0/1	Pending	0	2m
test-app-5d4d5b6c7b-k9p3n	0/1	Pending	0	2m
test-app-5d4d5b6c7b-x7v2q	0/1	Pending	0	2m

Check why Pods are pending. Describe a pending Pod to see its events. The output indicates `no scheduler found`:

```
$ kubectl describe pod test-app-5d4d5b6c7b-h2x4m | tail -10
```

Check the Pods in the `kube-system` namespace. The scheduler is in `ImagePullBackOff` state:

```
$ kubectl get pods -n kube-system | grep -E \
"scheduler|controller|apiserver|etcd"
```

Investigate the scheduler Pod. The scheduler is trying to use a non-existent image version:

```
$ kubectl describe pod kube-scheduler-master-node -n kube-system \
| grep -A5 Events
```

Fix the scheduler manifest. SSH into the control plane node. Edit the scheduler configuration manifest. Fix the scheduler manifest by changing the image tag to the version of control plane node, e.g., v1.33.2:

```
$ ssh control-plane
$ sudo vi /etc/kubernetes/manifests/kube-scheduler.yaml
```

Wait a moment for the kubelet to detect the change and restart the scheduler Pod. Once the scheduler Pod transitions back to normal operations, you can check for the `test-app` Pods again:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
test-app-5d4d5b6c7b-h2x4m	1/1	Running	0	8m
test-app-5d4d5b6c7b-k9p3n	1/1	Running	0	8m
test-app-5d4d5b6c7b-x7v2q	1/1	Running	0	8m

All Pods should be back to the `Running` status.